C++ Chapter 5: Functions

Contents

C++ Chapter 5: Functions 1
DRY 1
Online Tutorials 2
Functions 2
  Why Use Functions? 2
User Defined Functions 3
Function Prototypes 3
Tutorial 5.1: Void Function 3
Tutorial 5.2: Function with a Single Parameter 5
Tutorial 5.3: Function with Multiple Parameters 7
Tutorial 5.4: Function with Return Value 9
Tutorial 5.5: Function Overloading 12
Tutorial 5.6: Input Function with Try Catch 14
Tutorial 5.7: Recursion 16
Tutorial 5.8: Pass by Reference 19
Tutorial 5.9: Default Arguments 20
Tutorial 5.10: Build a C++ Program with Multiple Files 21
Assignment 1: Function Header Conversion Program 22
  Assignment Submission 22

Time required: 120 minutes

DRY

Don't **R**epeat **Y**ourself (**DRY**) is a principle of software engineering aimed at reducing repetition of software patterns. It you are repeating any code, there is probably a better solution.

Online Tutorials

Go through the following tutorials before going through the tutorial assignments.

- C++ Functions
- C++ Parameters/Arguments
- C++ Default Parameter
- C++ Multiple Parameters
- C++ Return Values
- C++ Function Overloading
- C++ Recursion

Functions

- Functions are building blocks
- Also called modules, methods, procedures, or sub-procedures
- Like miniature programs
- Can be put together to form larger program
- One of two principle means of modularization in C++ (other is classes)

Why Use Functions?

Divide and Conquer

- Allow complicated programs to be divided into manageable components
- Programmer can focus on just the function: develop it, debug it, and test it
- Various developers can work on different functions simultaneously

Reusability

- Can be used in more than one place in a program--or in different programs
- Avoids repetition of code, thus simplifying code maintenance
- Can be called multiple times from anywhere in the program

Components:

- Custom functions and classes that you write
- Prepackaged functions and classes available in the C++ Standard Library

User Defined Functions

There can be 4 different types of user-defined functions, they are:

- Function with no parameters and no return value
- Function with no parameters and a return value
- Function with parameters and no return value
- Function with parameters and a return value

Function Prototypes

A **function prototype** is a declaration of the function that tells the program about the type of the value which is to be returned by the function. It gives the number and type of parameters in the function. It allows the C++ compiler to find the function wherever it is in the program code.

The function prototype serves the following purposes –

1. Gives the return type of the data that the function will return.
2. Gives the number of arguments passed to the function.
3. Gives the data types of each of the passed arguments.
4. Gives the order in which the arguments are passed to the function.

The function prototype specifies the input/output interlace to the function i.e. what to give to the function and what to expect from the function.

The prototype of a function is also called the signature of the function.

Tutorial 5.1: Void Function

Time to create our first function. This program has two functions, **message()** and **main()**. All C++ program have a **main()** function where the program starts.

Create a C++ file with the name **VoidFunction.cpp**

Replace with your own favorite saying.

```cpp
/**
 * Name: VoidFunction.cpp
 * Written by:
 * Written on:
 * Purpose: Void function
 */

#include <iostream>
// Function prototype
void message();

int main()
{
    std::cout << "\nFirst function call" << std::endl;
    // Function call
    message();
    std::cout << "\nSecond function call" << std::endl;
    // Function call
    message();
    std::cout << "\nThird function call" << std::endl;
    // Function call
    message();
    return 0;
}

// Void function
void message()
{
    // Split a cout statement and hard coded string
    // across multiple lines for readability
    std::cout
        << "I love deadlines. "
        << "I like the whooshing sound they make as they fly by."
        << std::endl;

    std::cout << "Douglas Adams" << std::endl;
}
```

Example run:

```
First function call
I love deadlines. I like the whooshing sound they make as they fly by.
Douglas Adams

Second function call
I love deadlines. I like the whooshing sound they make as they fly by.
Douglas Adams

Third function call
I love deadlines. I like the whooshing sound they make as they fly by.
Douglas Adams
```

Tutorial 5.2: Function with a Single Parameter

The following program has a parameter with a single parameter.

Create a C++ program named: **FunctionSingleParameter.cpp**

```cpp
/**
 * Name: FunctionSingleParameter.cpp
 * Written by:
 * Written on:
 * Purpose: Function with 1 double parameter
 */

#include <iostream>
// Import iomanip library for setprecision
#include <iomanip>
// Import Math library for sqrt method
#include <cmath>
// Function prototype
void findSquareRoot(double num);

int main()
{
    // Initialize variables to hard coded values
    double num1{4};
    double num2{8.9};
    double num3{112};

    // Function calls with 1 double argument
    findSquareRoot(num1);
    findSquareRoot(num2);
    findSquareRoot(num3);
    return 0;
}

/****************** FIND SQUARE ROOT *********************
 * void function with 1 double parameter
 */
void findSquareRoot(double num)
{
    double squareRoot;
    squareRoot = sqrt(num);
    // Long line of code broken up into multiple lines
    // setprecision is needed to set
    // 15 digits of precision for double in C++ cout
    std::cout
        << std::setprecision(15)
        << "The square root of "
        << num << " is: " << squareRoot
        << std::endl;
}
```

A double in C++ has 15 digits of precision. cout will normally truncate the display unless you use **setprecision**. **setprecision** is part of the **iomanip** library.

Example run without setprecision:

```
The square root of 4 is: 2
The square root of 8.9 is: 2.98329
The square root of 112 is: 10.583
```

Example run with setprecision:

```
The square root of 4 is: 2
The square root of 8.9 is: 2.98328677803526
The square root of 112 is: 10.5830052442584
```

Tutorial 5.3: Function with Multiple Parameters

The following program demonstrates a function with 2 datatypes and 3 parameters.

Create a C++ program named: **FunctionMultipleParameters.cpp**

```cpp
/**
 * Name: FunctionMultipleParameters.cpp
 * Written by:
 * Written on:
 * Purpose: function with 2 double and one string parameter
 */
#include <iostream>

// Function prototype
void multiply(double num1, double num2, std::string message);

int main()
{
   // Initialize variables
   std::string message = "\nFirst function call";
   double num1{2};
   double num2{3.29};
   double num3{4};
   double num4{313};

   // Function call with 2 double and 1 string parameter
   multiply(num1, num2, message);

   message = "\nSecond function call";
   // Function call with 2 double and 1 string parameter
   multiply(num3, num4, message);
   return 0;
}

/****************** MULTIPLY *********************************
 * void method with 3 parameters
 */
void multiply(double num1, double num2, std::string message)
{
   double product;
   product = num1 * num2;
   std::cout << message << std::endl;
   // Break cout statement at << for readibility
   std::cout << "The product of "
           << num1 << " * "
           << num2 << " is: "
           << product << std::endl;
}
```

Example run:

```
First function call
The product of 2 * 3.29 is: 6.58

Second function call
The product of 4 * 313 is: 1252
```

Tutorial 5.4: Function with Return Value

Functions can do some work and return a value.

Example:

```
/**
 * Name: FunctionAddTwoNumbers.cpp
 * Written by:
 * Written on:
 * Purpose: two parameters and one return
 */

#include <iostream>
// Function prototype with two int parameters and an int return value
int sum(int, int);

int main()
{
    int num1;
    int num2;
    int output;

    std::cout << "Enter a number";
    std::cin >> num1;
    std::cout << "Enter a number";
    std::cin >> num2;

    // Method call with two arguments and one return value
    output = sum(num1, num2);
    std::cout << num1 << " + " << num2 << " = " << output << std::endl;
    return 0;
}

// Two parameters coming in
int sum(int x, int y)
{
    // Return value
    return x + y;
}
```

Create a C++ program named: **FunctionReturnValue.cpp**

```cpp
/**
 * Name: FunctionReturnValue.cpp
 * Written by:
 * Written on:
 * Purpose: Function with a return value
 */
#include <iostream>
// Import cmath library for ceil method
#include <cmath>
// Function one double parameter one double return
double getCeil(double num);

int main()
{
    double num1{4.54};
    double num2{8.2};
    double numCeil;

    std::cout << "\nPrint function return value directly"
              << std::endl;
    // function call with 1 argument and return value
    std::cout << getCeil(num1) << std::endl;

    std::cout << "Assign return value to variable" << std::endl;
    // Function call with 1 argument, assign return value to variable
    numCeil = getCeil(num2);
    std::cout << numCeil;
    return 0;
}

// Function with 1 double parameter
// and 1 double return value
double getCeil(double num)
{
    double numCeil;
    // cmath.ceil() method rounds a floating point
    // up to the nearest integer value
    numCeil = ceil(num);
    return numCeil;
}
```

Example run:

```
Print function return value directly
5
Assign return value to variable
9
```

Tutorial 5.5: Function Overloading

Function overloading occurs when a function is defined multiple times with different signatures.

We will also introduce default parameters. If an argument is not passed, the default parameter value is used. This is only setup in the function prototype, it is not included in the function definition.

```cpp
/**
 * Name: FunctionOverloading.cpp
 * Written by:
 * Written on:
 * Purpose: Overload function with different signatures
 */
#include <iostream>
// Needed for stoi method to parse integer from string
#include <string>
// Function prototypes
// If this function is called without an argument, 24 is used
int getResult(int num = 24);
int getResult(int num1, int num2);
int getResult(int num, std::string value);
std::string getResult(std::string str1, std::string str2);

int main()
{
    // Call method with a single int argument
    int result = getResult(5);
    std::cout << result << std::endl;

    // Call method with no arguments, use the default parameter
    result = getResult();
    std::cout << result << std::endl;

    // Call method with two int arguments
    result = getResult(5, 4);
    std::cout << result << std::endl;

    // Call method with an int and a string argument
    result = getResult(5, "12");
    std::cout << result << std::endl;

    // Call method with two string arguements
    std::cout << getResult("Billy", "Idol") << std::endl;
    return 0;
}
```

```
int getResult(int num)
{
    return num * 2;
}

int getResult(int num1, int num2)
{
    return num1 * num2;
}

int getResult(int num1, std::string string_value)
{
    // std::stoi is a string library function
    // that parses an integer from a string
    return num1 * std::stoi(string_value);
}

std::string getResult(std::string str1, std::string str2)
{
    return str1 + " " + str2;
}
```

Example run:

```
10
48
20
60
Billy Idol
```

Tutorial 5.6: Input Function with Try Catch

This method takes a prompt string, does error checking and/or input validation, then returns a value.

Create a C++ program named: **InputFunction.cpp**

```cpp
/**
 * Name: InputFunction.cpp
 * Written by:
 * Written on:
 * Purpose: Recursive function for valid user input
 */
#include <iostream>
// Include string library for stod convert string to double
#include <string>
// Function prototpye
double getDouble(std::string prompt);

int main()
{
   double number;
   // Call method with String prompt parameter
   number = getDouble("Enter number: ");
   std::cout << "Number: " << number << std::endl;
}
```

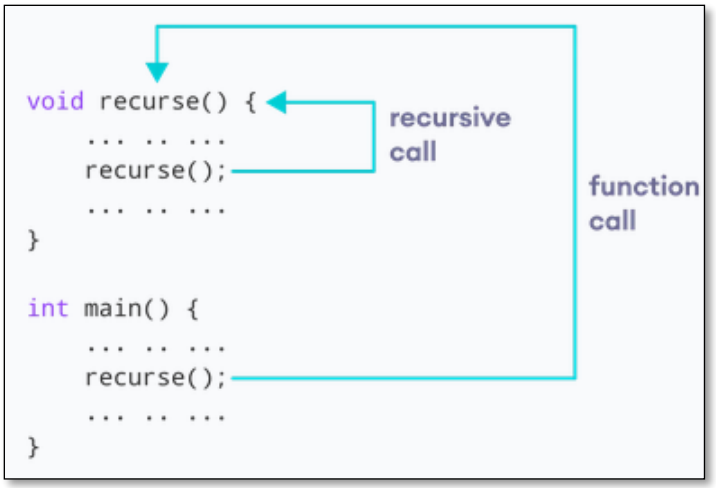
```
21 /*
22  * Prompt the user for input, return value
23  *
24  * @param String prompt the user for the value to be entered
25  */
26 double getDouble(std::string prompt)
27 {
28    std::string lineInput;
29    double doubleValue{0};
30    // Print the prompt parameter
31    std::cout << prompt;
32    // Get the input line into a string
33    getline(std::cin, lineInput);
34    // Catch any input mismatch exceptions
35    try
36    {
37       // Convert string to a double
38       doubleValue = std::stod(lineInput);
39       // Return value
40       return (doubleValue);
41    }
42    // Catch any exceptions
43    catch (...)
44    {
45       // Return method will recurse until the user enters a number
46       return (getDouble("Enter a number: "));
47    }
48 }
```

Example run:

```
Enter number: w
Enter a number: d
Enter a number: 56
Number: 56
```

Tutorial 5.7: Recursion

A function can call other functions or itself. Recursion is when a function calls itself to accomplish a task. Recursion should be used sparingly.


void recurse() {
... .
recurse();
... .
}
int main() {
... .
recurse();
... .
}
recursive call
function call

```cpp
/**
 * Name: CountDown.cpp
 * Written by:
 * Written on:
 * Purpose: Recursion demo
 */
#include <iostream>
// Function prototypes allow functions to be placed in any order
void countDownFrom(int num);
void countUpTo(int from, int to);

int main()
{
    std::cout << "Countdown" << std::endl;
    countDownFrom(10);
    std::cout << std::endl;

    std::cout << "Countup" << std::endl;
    countUpTo(0, 10);
    std::cout << std::endl;
}

void countDownFrom(int num)
{
    if (num >= 0)
    {
        std::cout << num << " ";
        // Resursive call to itself
        countDownFrom(num - 1);
    }
}

void countUpTo(int from, int to)
{
    if (from <= to)
    {
        std::cout << from << " ";
        // Resursive call to itself
        countUpTo(from + 1, to);
    }
}
```

Example run:

```
Countdown
10 9 8 7 6 5 4 3 2 1 0
Countup
0 1 2 3 4 5 6 7 8 9 10
```

Tutorial 5.8: Pass by Reference

We have been passing arguments by copying the value of the variable. The original variable is unchanged. A reference variable is a "reference" to an existing variable, and it is created with the & operator. The & operator refers to the memory location of the variable, not the name.

```cpp
/**
 * Filename: PassByReference.cpp
 * Written by:
 * Written on:
 * Purpose: & pass by reference
 */

#include <iostream>

void aFunction(int &num1, int &num2);

int main()
{
    int num1 = 5;
    int num2 = 7;
    aFunction(num1, num2);
    std::cout << num1 << ", " << num2 << std::endl;
    return 0;
}

void aFunction(int &num1, int &num2)
{
    // Pass by reference with &
    // This is a memory location of the original variable
    // The original variable is changed
    // The name doesn't matter, these are positional arguments
    num1 = 2;
    num2 = 2;
}
```

Example run:

```
2, 2
```

Tutorial 5.9: Default Arguments

It is possible to call a function and not have to pass all the arguments.

Here is an example function prototype with a default argument. If the second argument is not passed, the value of repeats is 1.

```
greeting(string greet, int repeats = 1);
```

Create the following example of a program with default arguments.

```
/**
 * Name: function_default_arguments.cpp
 * Written by:
 * Written on:
 * Purpose: Demonstrate default arguments in C++
 */
#include <iostream>
// Function prototype includes the default parameter
void greeting(std::string greet, int repeats = 1);

int main(void)
{
    greeting("Hello", 4);
    // Function call with one argument, default argument will be used
    greeting("Good morning");
    greeting("Hola amigo!", 10);
    return 0;
}

// Function definition does not include the default parameter
void greeting(std::string greet, int repeats)
{
    for (int i = 0; i < repeats; i++)
        std::cout << greet << std::endl;
}
```

Example run:

```
Hello
Hello
Hello
Hello
Good morning
Hola amigo!
Hola amigo!
Hola amigo!
Hola amigo!
Hola amigo!
Hola amigo!
Hola amigo!
Hola amigo!
Hola amigo!
Hola amigo!
```

Tutorial 5.10: Build a C++ Program with Multiple Files

As our code becomes increasingly complex, it is helpful to organize our code into separate files.

This example demonstrates the separation of a function into separate files.

1. **my_function.h** – header file that contains the function declaration.
2. **my_function.cpp** - defines the function's implementation.
3. **main.cpp** - includes the header file and uses the function.

Create a separate folder for your MyFunction project.

main.cpp

```cpp
// Include the header file containing function declaration
#include "my_function.h"

int main() {
    // Call the function defined in my_function.cpp
    myFunction();
    return 0;
}
```

my_function.h

```cpp
#ifndef MYFUNCTION_H
#define MYFUNCTION_H

// Function declaration
void myFunction();

#endif // MYFUNCTION_H header guard
// Prevents the header from being imported more than once.
```

my_function.cpp

```cpp
#include <iostream>
// Include the header file containing function declaration
#include "my_function.h"

// Function definition
void myFunction()
{
    std::cout << "This is my function!" << std::endl;
}
```

How to compile multiple files from the command line.

```
g++ -o my_program.exe main.cpp my_function.cpp
```

-o <output> specifies the output file name. For example, **-o my_program.exe** sets the output file name as **my_program.exe**

Example run:

```
This is my function!
```

Assignment 1: Function Header Conversion Program

Create a C++ program that does measurement conversion with separate function header, function and main files.

Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the program.
3. Submit in Blackboard.